EX NO : 01	Device ON / OFF using PIC 16F877A microcontroller (Relay and LED)	Batch No : 02
DATE : 16/07/2025		Roll No : 23ECR117

AIM :

- To blink an LED using the PIC16F877A microcontroller to demonstrate basic digital output control.
- To control a fan or light by switching a relay ON or OFF using a switch and the PIC16F877A microcontroller.

Software Required :

- PIC C Compiler to compile and create hex file.
- PICkit 2 programmer for burning the hex file to PIC microcontroller.
- Proteus 8.17 is used for the simulation of circuits.

Apparatus Required :

Sl. No	Apparatus name	Range	Quantity
1.	Bread board	-	1
2.	Regulated Power Supply	5V, 2A	1
3.	PIC16F877A	40 - Pin PDIP	1
4.	Crystal oscillator	4 MHz	1
5.	PIC programmer and USB cable	-	1
6.	Light Emitting Diode (LED)	-	1
7.	Switch	SPST / SPDT	1
8.	Relay	12V / 5V	1
9.	BC547 - NPN Transistor	-	1
10.	Diode	1N4001	1
11.	Resistor	330 Ω,1k Ω	2 Each
12.	Connecting wires	Single strand	As required

Procedure :

- Open PIC C Compiler (CCS).
- Create a New Project using the Project Wizard.
- Select PIC16F877A from the PIC16 family.
- Set the crystal frequency to 4 MHz.
- Select the I/O pins and define each as Input, Output, Input/Output, or None as required.
- In the Fuses settings, tick the checkbox for Power-up Timer.
- Write the required C code and save the project.
- Click Build/Compile to generate the .hex file.

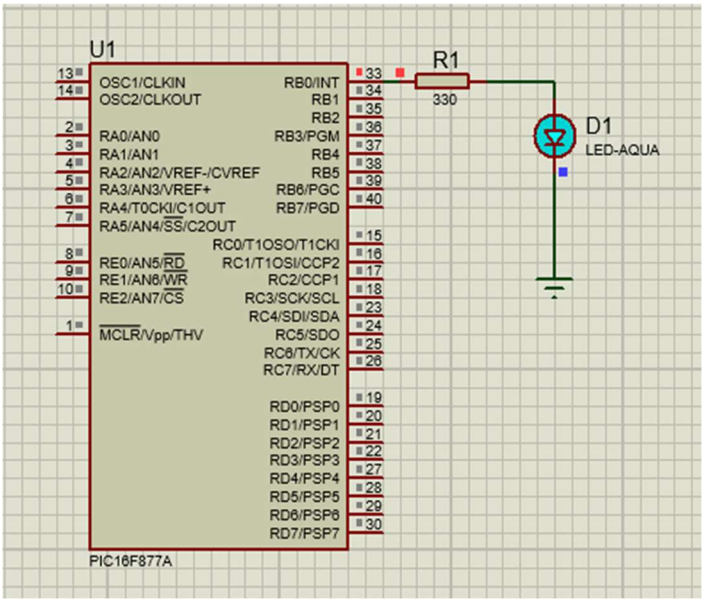
- Open Proteus, design the circuit with PIC, LED, switch, relay, etc.
- Double-click the PIC in Proteus and load the compiled .hex file.
- Click Run to simulate and observe the output.

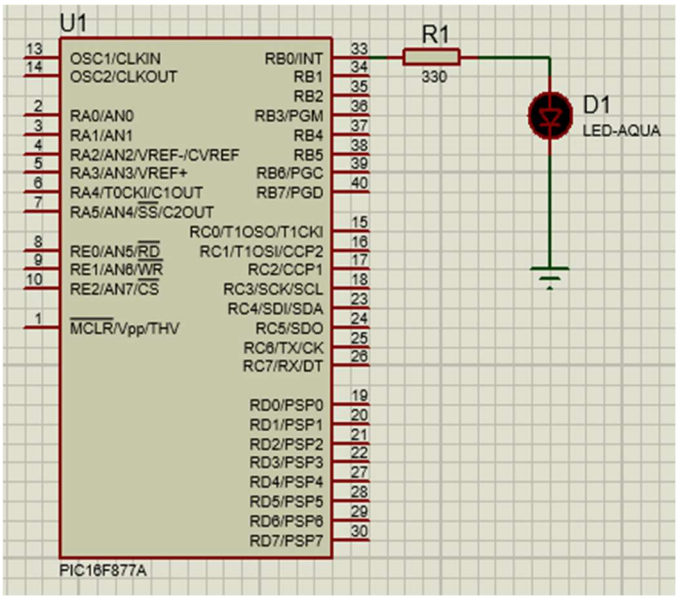
1) Blinking of LED using PIC 16F877A Microcontroller :

Code :

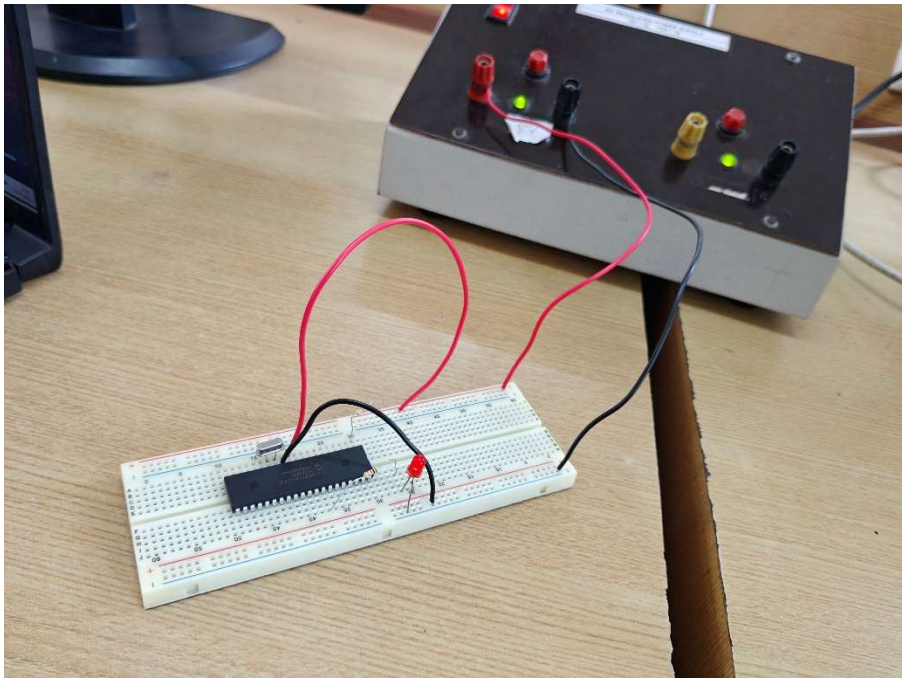
```
#include <16F877A.h>
#device ADC=16

#FUSES NOWDT                    //No Watch Dog Timer
#FUSES PUT                      //Power Up Timer
#FUSES NOBROWNOUT               //No brownout reset
#FUSES NOLVP                    //No low voltage prgming, B3(PIC16) or B5(PIC18) used for I/O

#use delay(crystal=4MHz)
#use FIXED_IO( B_outputs=PIN_B0 )
#define LED   PIN_B0

void main()
{

  while(TRUE)
  {
    output_high(LED);
     delay_ms(100);

     output_low(LED);
     delay_ms(100);
  }

}
```

Simulation output :





Hardware Output :

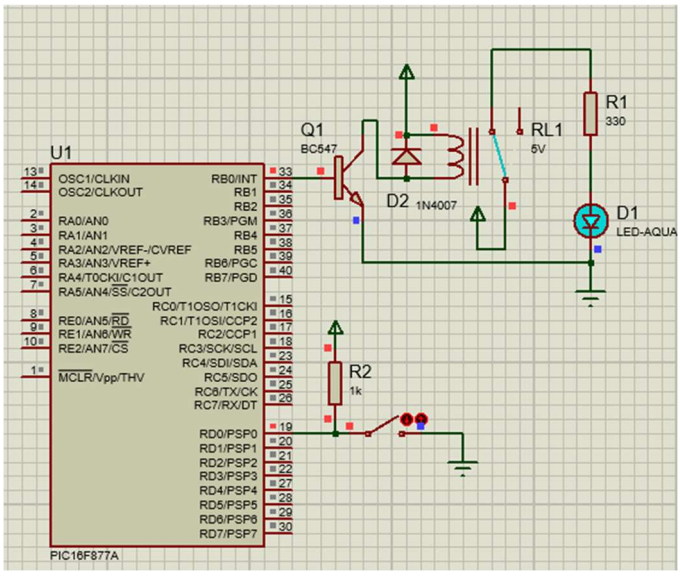


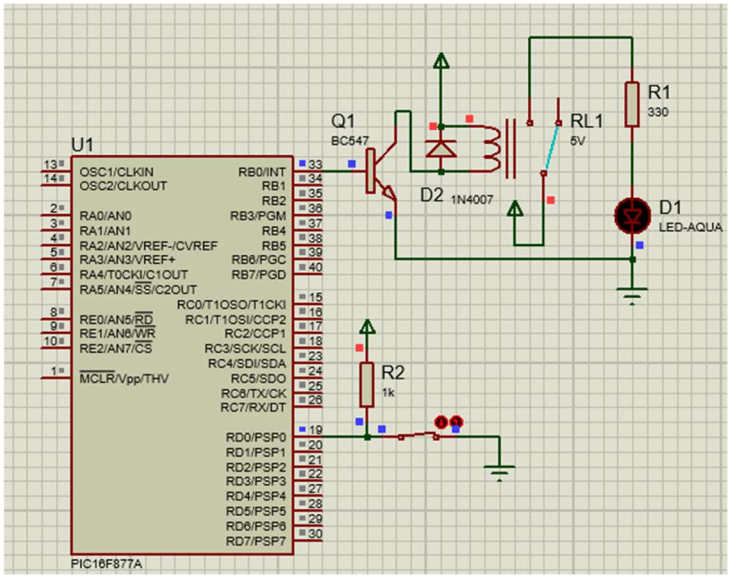
2) LED ON/OFF using Relay :

Code :

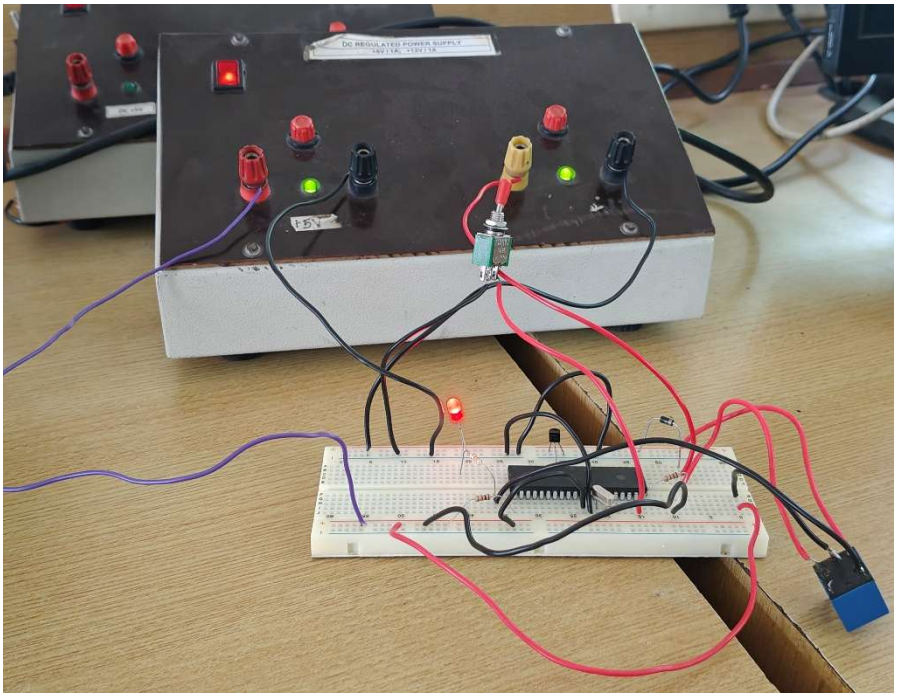
```
#include <16F877A.h>
#device ADC=16

#FUSES NOWDT                    //No Watch Dog Timer
#FUSES PUT                      //Power Up Timer
#FUSES NOBROWNOUT               //No brownout reset
#FUSES NOLVP                    //No low voltage prgming, B3(PIC16) or B5(PIC18) used for I/O

#use delay(crystal=4MHz)
#use FIXED_IO( B_outputs=PIN_B0 )
#define LED   PIN_B0
#define SWITCH1   PIN_D0

void main()
{

  while(TRUE)
  {
    if (input(SWITCH1) == 0)
       output_low(LED);
     else
       output_high(LED);
  }

}
```

Simulation output :





Hardware Output :



QR Code:

1) Led Blink :

2) Relay :

Rubrics		Marks
Conduct of Experiment	Analyse the problem and develop programming constructs (15)	
	Completeness of the experiment (5)	
Observation/ Record (30)	Interpretation of the findings (15)	
	Simulation and Hardware (5)	
	Adherence to record submission deadline (5)	
	Presentation and completion of record (5)	
Viva (10)	Ability to recall the theoretical concepts	
Total (60)		

Result :

Thus the Device ON / OFF using PIC 16F877A microcontroller (Relay and LED) was is done successfully by using PIC C Compiler and Proteus Software 8.17.

EX NO : 02	Interfacing of LCD with PIC 16F877A microcontroller	Batch No : 02
DATE : 29/07/2025		Roll No : 23ECR117

AIM :

To interface a 16x2 LCD with the PIC 16F877A microcontroller and display a message on the LCD using the PIC C Compiler.

Software Required :

- PIC C Compiler to compile and create hex file.
- PICkit 2 programmer for burning the hex file to PIC microcontroller.
- Proteus 8.17 is used for the simulation of circuits.

Apparatus Required :

S. No	Apparatus name	Range	Quantity
1.	Bread board	-	1
2.	Regulated Power Supply	5V, 2A	1
3.	PIC16F877A	40-Pin PDIP	1
4.	Crystal oscillator	4MHz	1
5.	PIC programmer and USB cable	-	1
6.	LCD Display	16X2	1
7.	Resistor	1K	1
8.	Connecting wires	Single strand	As required

Procedure :

- Open PIC C Compiler (CCS).
- Create a New Project using the Project Wizard.
- Select PIC16F877A from the PIC16 family.
- Set the crystal frequency to 4 MHz.
- Select the LCD(External) and define each as A0..,B0..,C0..,D0..,E0.., or None as required.
- In the Fuses settings, tick the checkbox for Power-up Timer.
- Write the required C code and save the project.
- Click Build/Compile to generate the .hex file.
- Open Proteus, design the circuit with LCD.
- Double-click the PIC in Proteus and load the compiled .hex file.
- Click Run to simulate and observe the output.

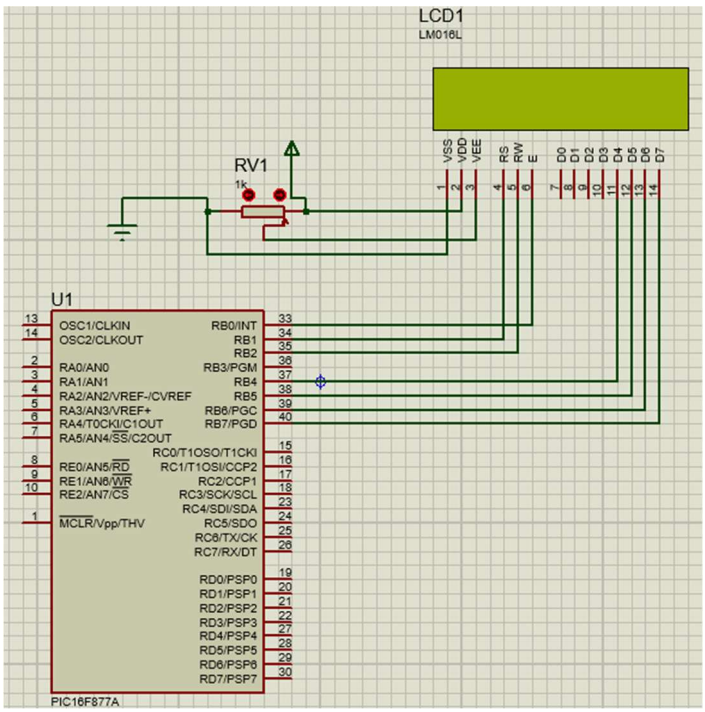
LCD with PIC 16F877A microcontroller :

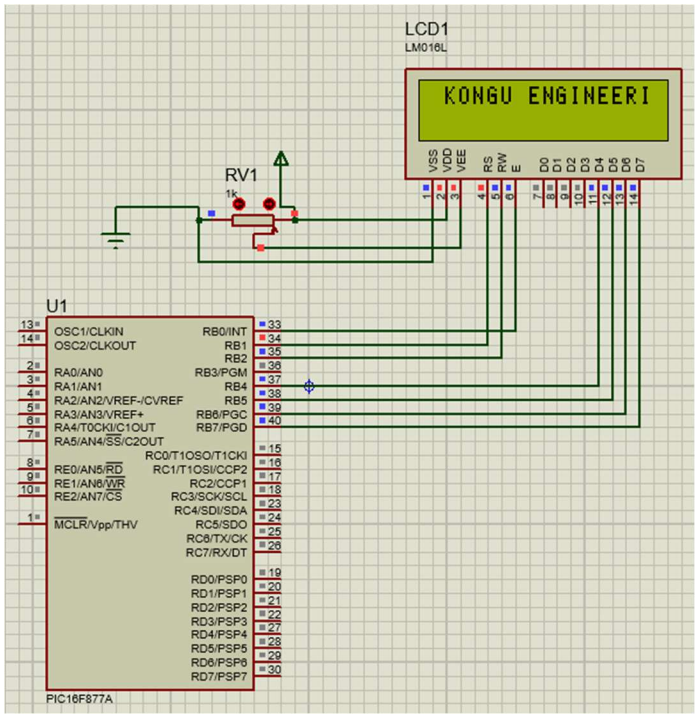
Code :

```
#include <16F877A.h>

#device ADC=16


#FUSES NOWDT
```

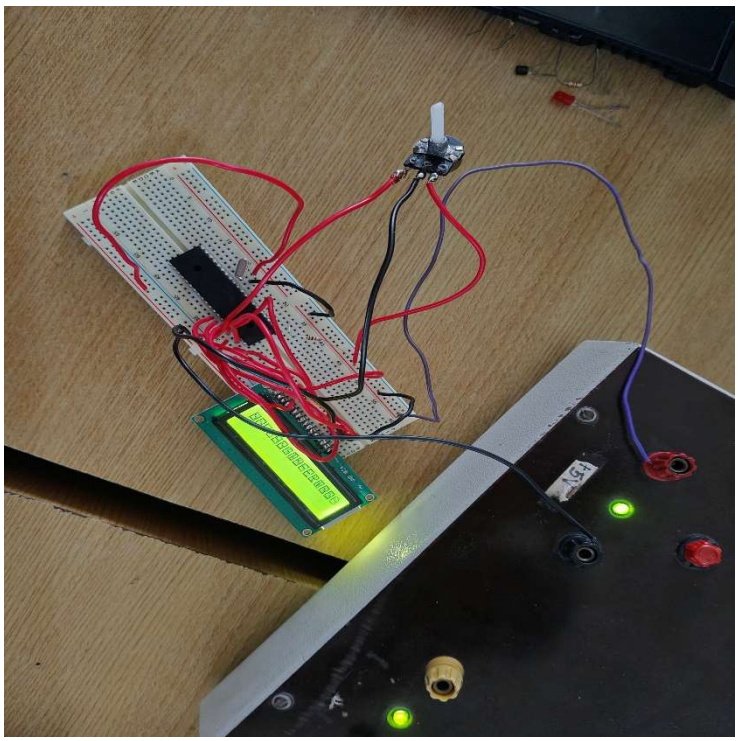
```
#FUSES PUT
#FUSES NOBROWNOUT
#FUSES NOLVP

#use delay(crystal=4MHz)

#define LCD_ENABLE_PIN PIN_B0
#define LCD_RS_PIN PIN_B1
#define LCD_RW_PIN PIN_B2
#define LCD_DATA4 PIN_B4
#define LCD_DATA5 PIN_B5
#define LCD_DATA6 PIN_B6
#define LCD_DATA7 PIN_B7

#include <lcd.c>

void main()
{
  char msg[] = "  KONGU ENGINEERING COLLEGE PERUNDURAI  ";
  int i = 0;

  lcd_init();

  while(TRUE)
  {
    for(i = 0; i < 23 ; i++ )
    {
      lcd_gotoxy(1, 1);
      printf(lcd_putc, "%s", msg + i);
      delay_ms(400);
    }
  }
}
```

Simulation output :





Hardware Output :



QR Code:

Rubrics		Marks
Conduct of Experiment	Analyse the problem and develop programming constructs (15)	
	Completeness of the experiment (5)	
Observation/ Record (30)	Interpretation of the findings (15)	
	Simulation and Hardware (5)	
	Adherence to record submission deadline (5)	
	Presentation and completion of record (5)	
Viva (10)	Ability to recall the theoretical concepts	
Total (60)		

Result :

The message " KONGU ENGINEERING COLLEGE PERUNDURAI " was successfully displayed on the 16x2 LCD, proving successful interfacing with the PIC16F877A microcontroller.

EX NO : 03	Analog sensor interfacing with PIC16F877A microcontroller	Batch No : 02
DATE : 06/08/2025		Roll No : 23ECR117

AIM :

To interface an analog sensor with the PIC16F877A microcontroller, read analog input values using the internal ADC, and display the digital and corresponding analog voltage values on a 16x2 LCD.

Software Required :

- PIC C Compiler to compile and create hex file.
- PICkit 2 programmer for burning the hex file to PIC microcontroller.
- Proteus 8.17 is used for the simulation of circuits.

Apparatus Required :

S. No	Apparatus name	Range	Quantity
1.	Bread board	-	1
2.	Regulated Power Supply	5V, 2A	1
3.	PIC16F877A	40-Pin PDIP	1
4.	Crystal oscillator	4MHz	1
5.	PIC programmer and USB cable	-	1
6.	Analog Sensor	-	1
7.	LCD Display	16X2	1
8.	Potentiometer	10K	2
9.	Resistor	1K	2
10.	Switch on/off	-	1
11.	Connecting wires	Single strand	As required

Procedure :

- Open PIC C Compiler (CCS).
- Create a New Project using the Project Wizard.
- Select **PIC16F877A** from the PIC16 family.
- Set the crystal frequency to **4 MHz**.
- Select the LCD (External) and assign pins as **B0–B2 (control)** and **B4–B7 (data)**.
- In the Fuses settings, tick the checkbox for **Power-up Timer.**
- Write the C code to read analog input (AN0/AN1) and display on LCD.
- Click **Build/Compile** to generate the **.hex file**.
- Open **Proteus**, design the circuit with **PIC, LCD, sensor**, and **switch**.
- Double-click the PIC in Proteus and load the compiled **.hex** file.
- Click Run to simulate and observe the **ADC value and voltage** on LCD.

Analog sensor interfacing with PIC16F877A :

Code :

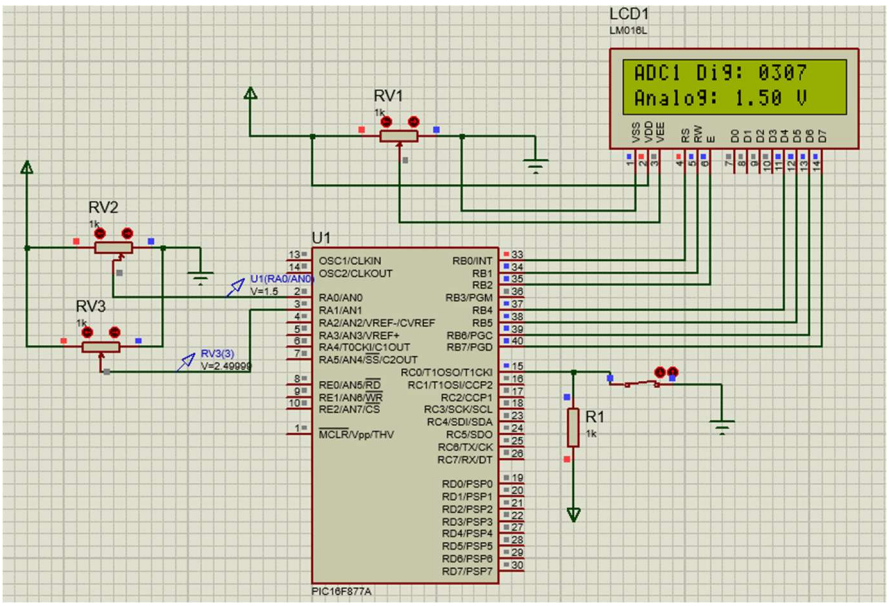
```
#include <16F877A.h>

#device ADC=10
```

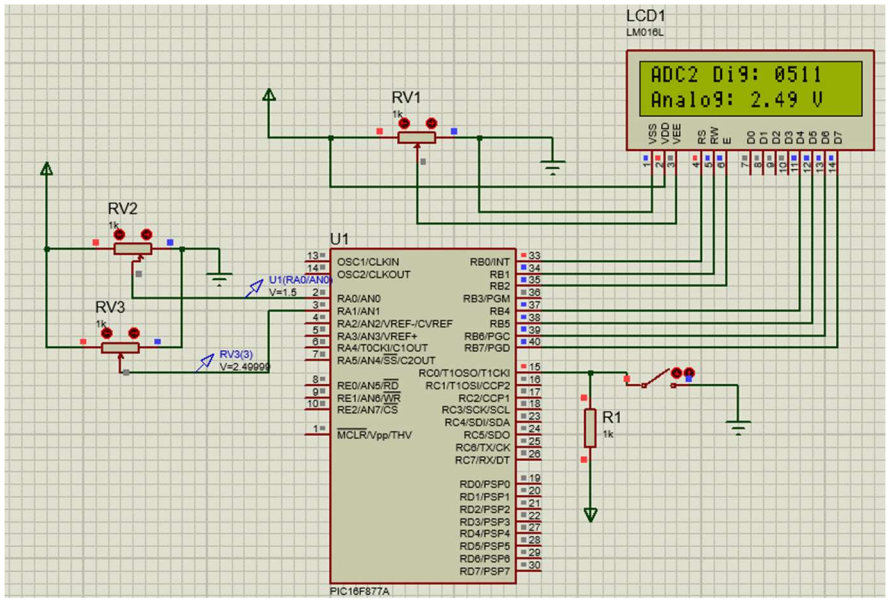
```
#FUSES NOWDT          // No Watch Dog Timer
#FUSES PUT            // Power Up Timer
#FUSES NOBROWNOUT     // No brownout reset
#FUSES NOLVP          // No low voltage programming

#use delay(crystal=4MHz)
#use FIXED_IO(C_outputs=PIN_C0)

#define LCD_ENABLE_PIN PIN_B2
#define LCD_RS_PIN     PIN_B0
#define LCD_RW_PIN     PIN_B1
#define LCD_DATA4      PIN_B4
#define LCD_DATA5      PIN_B5
#define LCD_DATA6      PIN_B6
#define LCD_DATA7      PIN_B7

#define SWITCH1        PIN_C0

#include <lcd.c>

void main() {
  int16 value;
  float voltage;



lcd_init();
 setup_adc_ports(ALL_ANALOG);
 setup_adc(ADC_CLOCK_INTERNAL);

while (TRUE) {
 if (input(SWITCH1) == 0) {

   set_adc_channel(0);
  delay_ms(10);

   read_adc(ADC_START_ONLY);
```

```
  int1 done = adc_done();

  while(!done) {
   done = adc_done();
    }

    value = read_adc(ADC_READ_ONLY);
    voltage = (value * 5.0) / 1023.0;

   lcd_gotoxy(1, 1);
    printf(lcd_putc, "ADC1 Dig: %04Lu", value);

lcd_gotoxy(1, 2);
 printf(lcd_putc, "Analog: %1.2f V ", voltage);
}
 else
  {
  set_adc_channel(1);
 delay_ms(10);

read_adc(ADC_START_ONLY);
 int1 done = adc_done();

 while(!done) {
    done = adc_done();
   }

  value = read_adc(ADC_READ_ONLY);
 voltage = (value * 5.0) / 1023.0;

  lcd_gotoxy(1, 1);
  printf(lcd_putc, "ADC2 Dig: %04Lu", value);

lcd_gotoxy(1, 2);
    printf(lcd_putc, "Analog: %1.2f V ", voltage);
  }
```

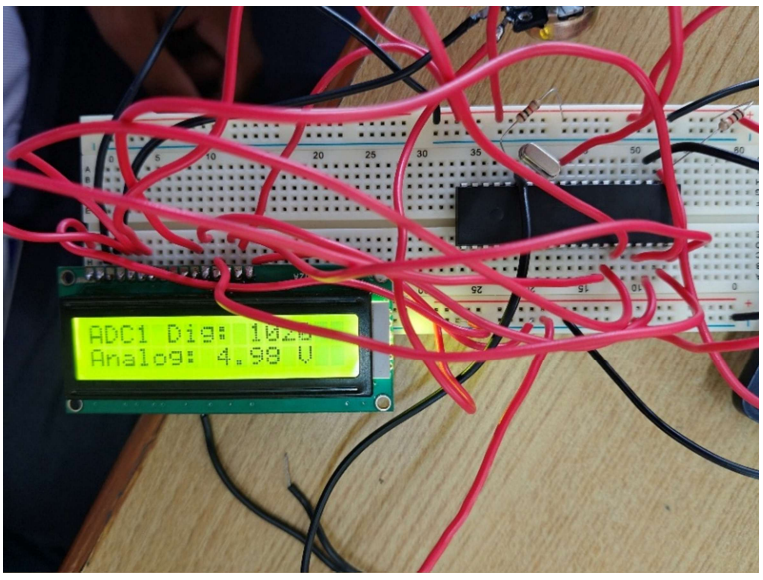
```
        delay_ms(200);
      }
    }
```

Simulation output :





Hardware Output :



QR Code :

Rubrics		Marks
Conduct of Experiment	Analyse the problem and develop programming constructs (15)	
	Completeness of the experiment (5)	
Observation/ Record (30)	Interpretation of the findings (15)	
	Simulation and Hardware (5)	
	Adherence to record submission deadline (5)	
	Presentation and completion of record (5)	
Viva (10)	Ability to recall the theoretical concepts	
Total (60)		

Result :

The analog sensor was successfully interfaced with the PIC16F877A microcontroller. The ADC module converted the analog signal into digital values and displayed the corresponding voltage on the LCD.

EX NO : 04	Design of clock using Real Time Clock with PIC 16F877A Microcontroller	Batch No : 02
DATE : 03/09/2025		Roll No : 23ECR117

AIM :

To Design and Implement a digital clock system using the PIC16F877A microcontroller with a Real Time Clock (RTC) module.

Software Required :

- PIC C Compiler to compile and create hex file.
- PICkit 2 programmer for burning the hex file to PIC microcontroller.
- Proteus 8.17 is used for the simulation of circuits.

Apparatus Required :

S. No	Apparatus Name	Range / Specification	Quantity
1	Breadboard	–	1
2	Regulated Power Supply	5V, 2A	1
3	PIC Microcontroller	PIC16F877A, 40-pin PDIP	1
4	Crystal Oscillator	4 MHz	1
5	PIC Programmer + USB Cable	–	1
6	RTC IC	DS1307	1
7	Crystal Oscillator (for RTC)	32.768 kHz	1
8	LCD Display	16x2 (LM032L)	1
9	Potentiometer	10K	1
10	Resistors	1K	2
11	Switch (ON/OFF)	–	1
12	Connecting Wires	Single strand	As required

Procedure :

- Open PIC C Compiler (CCS) and create a new project using the Project Wizard.
- Select the **PIC16F877A** microcontroller from the PIC16 family.
- Set the crystal frequency to 4 MHz.
- Select LCD (External) and assign control pins as B0–B2 and data pins as B4–B7.
- Enable **I²C communication** (SCL → RC3, SDA → RC4) for DS1307 RTC.
- In the fuses settings, tick the Power-up Timer option.
- Write the C code to initialize LCD and I²C, read time/date from **DS1307**, convert to decimal, and display on LCD.
- Click Build/Compile to generate the **.hex** file.

- Open Proteus and design the circuit with PIC, DS1307 RTC, 32.768 kHz crystal, battery, pull-up resistors, and LCD.
- Double-click the PIC in Proteus and load the **compiled .hex** file.
- Click Run to simulate and observe the real-time clock values **(HH:MM:SS and DD/MM/YY)** on LCD.

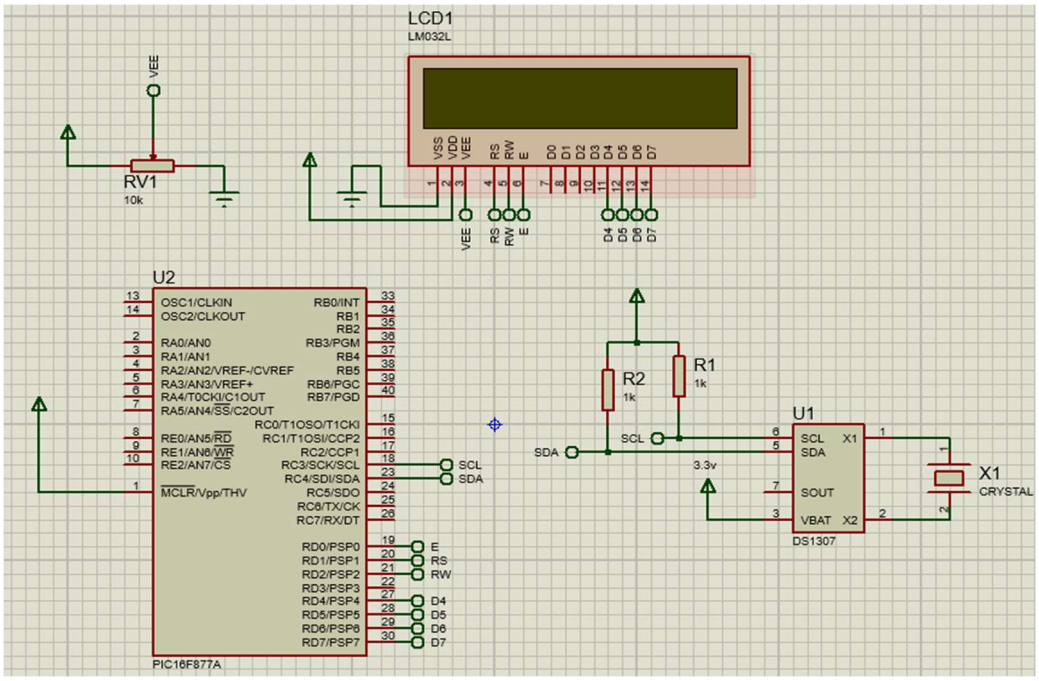
Real Time Clock :

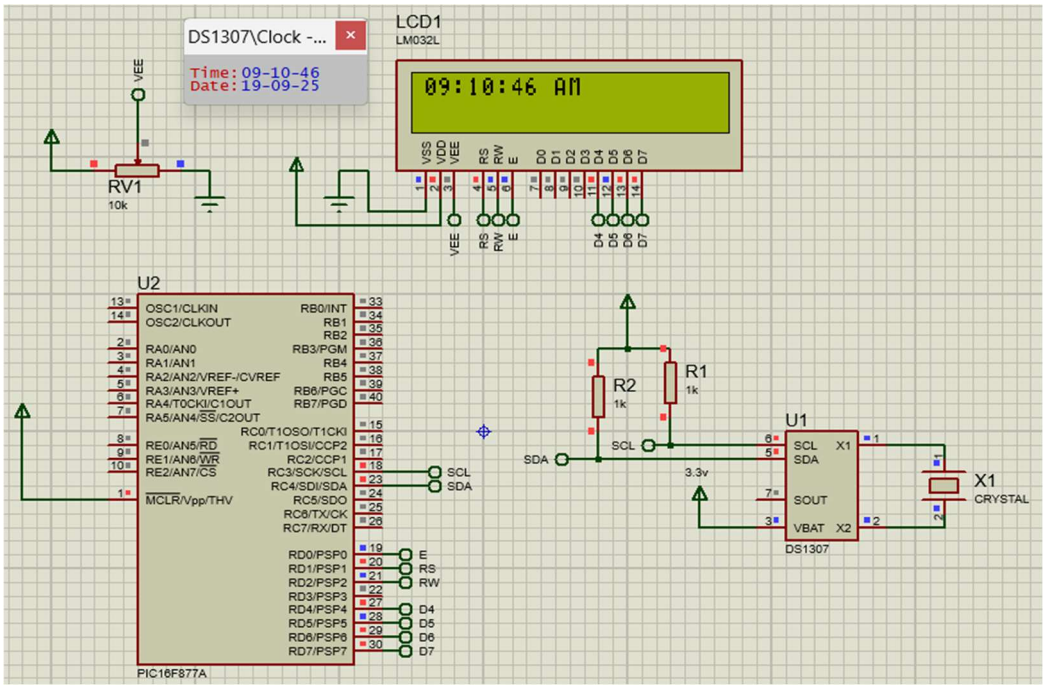
Code :

```
#include <16F877A.h>
#device ADC=10
// Fuses
#FUSES NOWDT        // No Watch Dog Timer
#FUSES PUT          // Power Up Timer
#FUSES NOBROWNOUT   // No brownout reset
#FUSES NOLVP        // No low voltage programming
#use delay(crystal=4MHz)
#use i2c(Master, Fast, sda=PIN_C4, scl=PIN_C3)

// LCD pin configuration
#define LCD_ENABLE_PIN  PIN_D0
#define LCD_RS_PIN      PIN_D1
#define LCD_RW_PIN      PIN_D2
#define LCD_DATA4       PIN_D4
#define LCD_DATA5       PIN_D5
#define LCD_DATA6       PIN_D6
#define LCD_DATA7       PIN_D7
#include <lcd.c>

#define bcd_to_dec(x)  (((x >> 4) * 10) + (x & 0x0F))

void main() {
   int sec, min, hrs;
   char ampm[3];   // "AM" or "PM"
   lcd_init();

   while(TRUE) {
      i2c_start();
      i2c_write(0xD0);
      i2c_write(0x00);
```

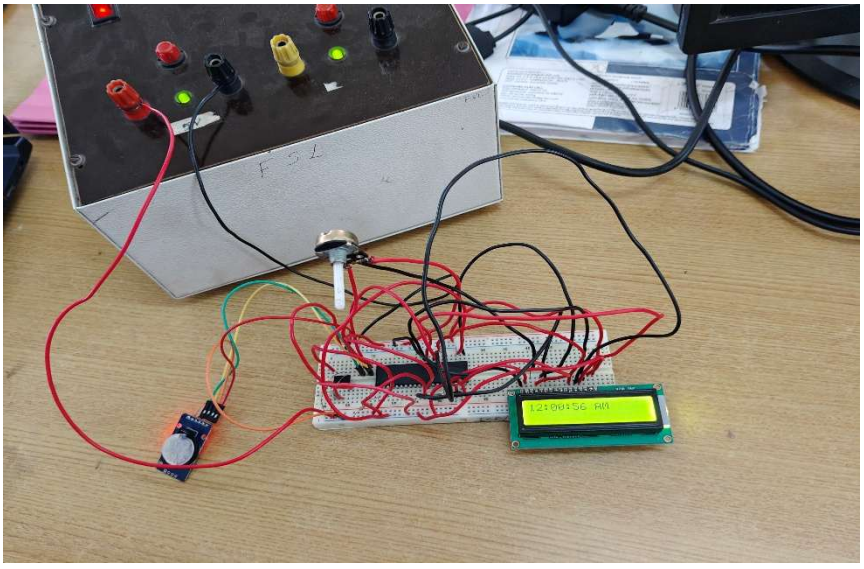
```
        i2c_start();
        i2c_write(0xD1);

        sec = i2c_read(1);
        min = i2c_read(1);
        hrs = i2c_read(0);

        i2c_stop();

        // Convert BCD to Decimal
        sec = bcd_to_dec(sec);
        min = bcd_to_dec(min);
        hrs = bcd_to_dec(hrs);

        // 24-hour to 12-hour conversion
        if(hrs == 0) {
           hrs = 12;
           strcpy(ampm, "AM");
        }
        else if(hrs < 12) {
           strcpy(ampm, "AM");
        }
        else if(hrs == 12) {
           strcpy(ampm, "PM");
        }
        else {
           hrs = hrs - 12;
           strcpy(ampm, "PM");
        }

        // Display on LCD
        lcd_gotoxy(1,1);
        printf(lcd_putc, "%02d:%02d:%02d %s", hrs, min, sec, ampm);

        delay_ms(1000);
    }
}
```

Simulation output :





Hardware Output :



QR Code :

Rubrics		Marks
Conduct of Experiment	Analyse the problem and develop programming constructs (15)	
	Completeness of the experiment (5)	
Observation/ Record (30)	Interpretation of the findings (15)	
	Simulation and Hardware (5)	
	Adherence to record submission deadline (5)	
	Presentation and completion of record (5)	
Viva (10)	Ability to recall the theoretical concepts	
Total (60)		

Result :

Thus the Design and Implementation of a digital clock system using the PIC16F877A microcontroller with a Real Time Clock (RTC) module has been done successfully.

EX NO : 05	Control LEDs via Classic Bluetooth using the WDM board (Bluetooth Communication)	Batch No : 02
DATE : 17/09/2025		Roll No : 23ECR117

AIM :

To design and implement a code to control and blink led via Classic Bluetooth using the WDM board and ThingZKit Mini.

Software Required :

Arduino IDE

Hardware Required :

ThingZMate Kit,USB cable.

Procedure :

- Open Arduino IDE and create a new sketch.
- Include the BluetoothSerial.h library for Bluetooth communication.
- Define the device name as **ESP32-BT-PANDA** and set GPIO2 as the LED pin.
- In setup(), initialize Serial at 9600 baud, start Bluetooth, and set LED pin as output (initially OFF).
- In loop(), check for Bluetooth data:
  - If 'h' or 'H' is received → turn LED ON.
  - If 'l' or 'L' is received → turn LED OFF.
- Send confirmation messages to Serial Monitor and Bluetooth terminal.
- Upload the code to ESP32 and pair it with a phone/computer via Bluetooth.
- Open a Bluetooth terminal app, connect to **ESP32-BT-PANDA**, and test by sending **'h'** and **'l'**.
- Observe LED ON/OFF status and messages on Serial Monitor and Bluetooth terminal.

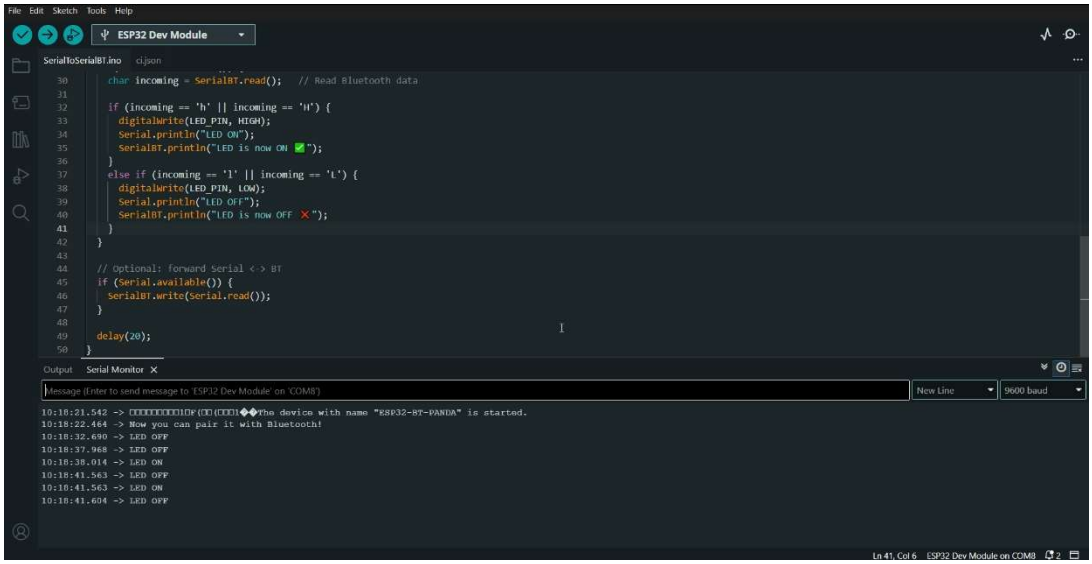
Bluetooth Communication :

Code :

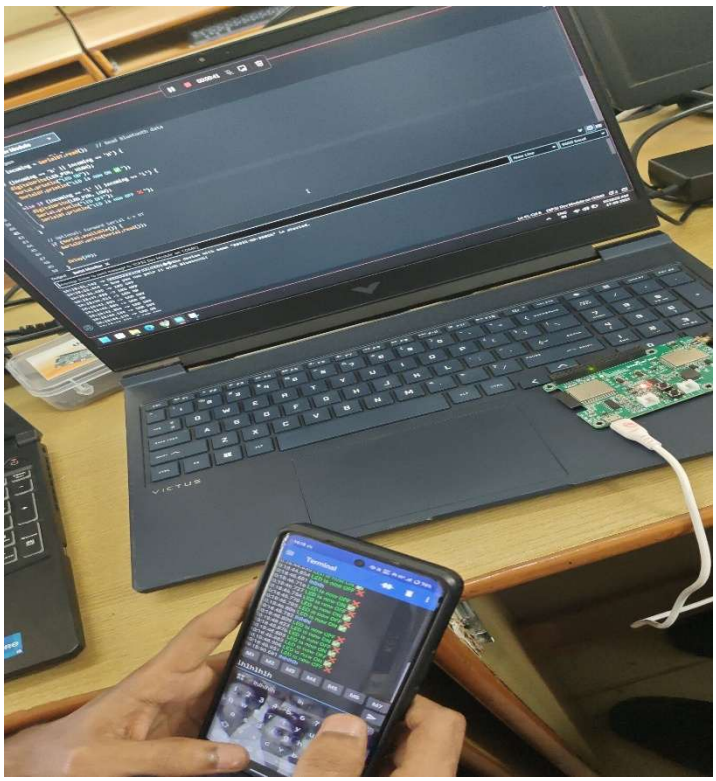
```
#include "BluetoothSerial.h"

String device_name = "ESP32-BT-PANDA";

#if !defined(CONFIG_BT_ENABLED) || !defined(CONFIG_BLUEDROID_ENABLED)

#error Bluetooth is not enabled! Please run `make menuconfig` to enable it

#endif


#if !defined(CONFIG_BT_SPP_ENABLED)

#error Serial Port Profile for Bluetooth is not available or not enabled.

#endif
```

```
BluetoothSerial SerialBT;

#define LED_PIN 2   // LED on D2 (GPIO2)
void setup() {
  pinMode(LED_PIN, OUTPUT);
  digitalWrite(LED_PIN, LOW);  // Start with LED OFF

  Serial.begin(9600);
  SerialBT.begin(device_name);
  Serial.printf("The device with name \"%s\" is started.\nNow you can pair it with Bluetooth!\n", device_name.c_str());
  SerialBT.println("ESP32 Bluetooth LED Control Ready!");
}

void loop() {
  if (SerialBT.available()) {
    char incoming = SerialBT.read();   // Read Bluetooth data

    if (incoming == 'h' || incoming == 'H') {
      digitalWrite(LED_PIN, HIGH);
      Serial.println("LED ON");
      SerialBT.println("LED is now ON ✅");
    }
    else if (incoming == 'l' || incoming == 'L') {
      digitalWrite(LED_PIN, LOW);
      Serial.println("LED OFF");
      SerialBT.println("LED is now OFF ❌");
    }
  }

  // Optional: forward Serial <-> BT
  if (Serial.available()) {
    SerialBT.write(Serial.read());
  }

  delay(20);
}
```

Arduino IDE Serial Monitor output :



Hardware Output :



QR Code :

Hardware

Arduino IDE Serial Monitor

Rubrics		Marks
Conduct of Experiment	Analyse the problem and develop programming constructs (15)	
	Completeness of the experiment (5)	
Observation/ Record (30)	Interpretation of the findings (15)	
	Simulation and Hardware (5)	
	Adherence to record submission deadline (5)	
	Presentation and completion of record (5)	
Viva (10)	Ability to recall the theoretical concepts	
Total (60)		

Result :

Thus the designing and implement of a code to control and blink led via classic bluetooth using the WDM board and ThingZKit Mini has been done successfully.

EX NO : 06	To Push sensor data to a cloud platform and create a dashboard via the HTTP protocol using the WDM board.	Batch No : 02
DATE : 24/09/2025		Roll No : 23ECR117

AIM :

To Push sensor data to a cloud platform and create a dashboard via the HTTP protocol using the WDM board.

Software Required :

Arduino IDE

Hardware Required :

ThingZMate Kit,USB cable

Procedure :

- Open Arduino IDE and create a new sketch.
- Include libraries: Wire.h, Adafruit_SHT31.h, WiFi.h, WiFiMulti.h, and HTTPClient.h.
- Initialize SHT31 sensor object and WiFi connection object.
- In setup():
  - Start Serial Monitor.
  - Initialize the SHT31 sensor.
  - Connect ESP32 to WiFi using SSID and Password.
- In loop():
  - Read temperature and humidity from the sensor.
  - Send the values to ThingSpeak using HTTP GET request with your API key.
  - Print the response on Serial Monitor.
  - Wait 5 seconds before the next reading.
- Upload the code to ESP32.
- Open Serial Monitor to check messages.
- Open your ThingSpeak channel and observe live graphs of temperature and humidity.

Wifi Communication :

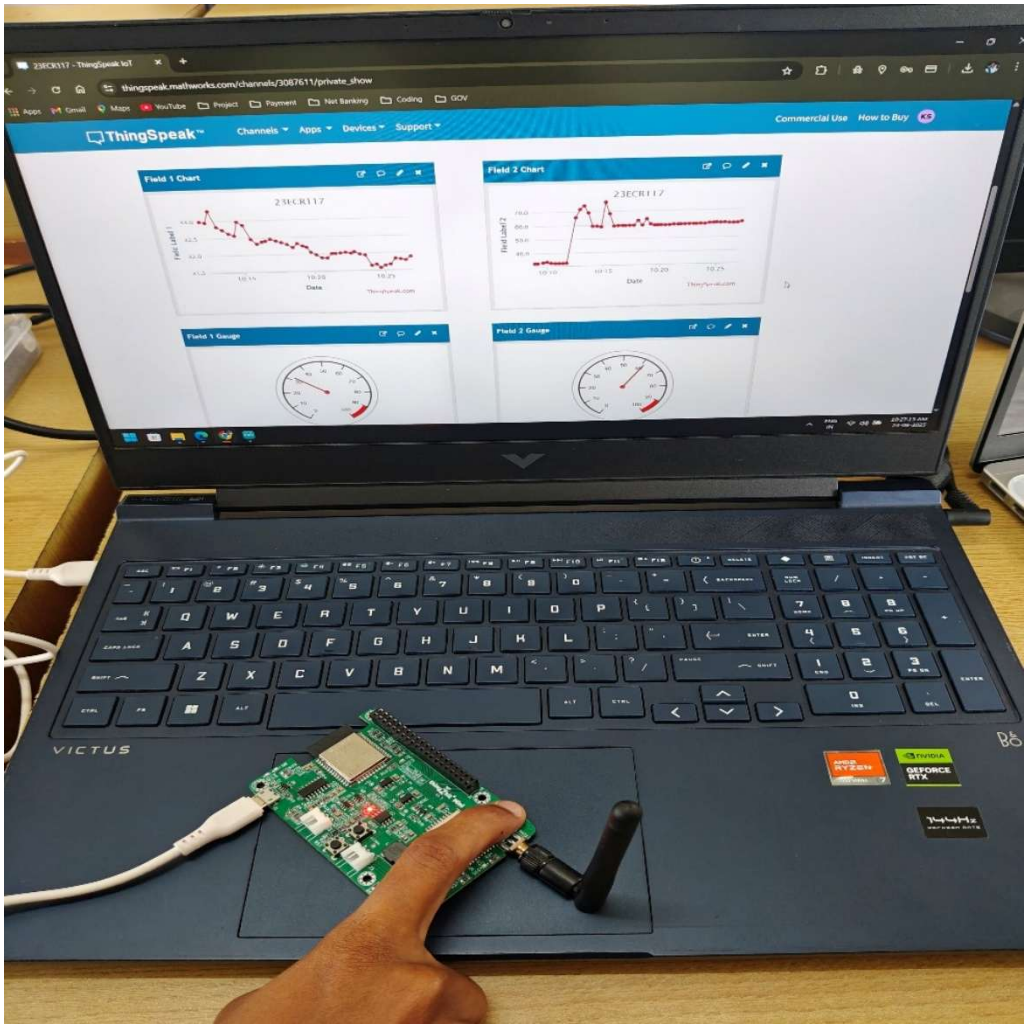
Code :

```
#include <Arduino.h>
#include <Wire.h>
#include "Adafruit_SHT31.h"
#include <WiFi.h>
```

```
#include <WiFiMulti.h>
#include <HTTPClient.h>
#define USE_SERIAL Serial

bool enableHeater = false;
uint8_t loopCnt = 0;

Adafruit_SHT31 sht31 = Adafruit_SHT31();
WiFiMulti wifiMulti;

void setup() {
Serial.begin(9600);
while (!Serial)
 delay(10);     // will pause Zero, Leonardo, etc until serial console opens

  Serial.println("SHT31 test");
 if (! sht31.begin(0x44)) {   // Set to 0x45 for alternate i2c addr
   Serial.println("Couldn't find SHT31");
   while (1) delay(1);
  }

 USE_SERIAL.begin(115200);

  USE_SERIAL.println();
  USE_SERIAL.println();
  USE_SERIAL.println();

  for (uint8_t t = 4; t > 0; t--) {
   USE_SERIAL.printf("[SETUP] WAIT %d...\n", t);
  USE_SERIAL.flush();
 delay(1000);
 }

wifiMulti.addAP("OnePlus Nord 4", "123456789");
}

void loop() {
 // wait for WiFi connection
  char buf[300];
 if ((wifiMulti.run() == WL_CONNECTED)) {
```

```
 HTTPClient http;

 USE_SERIAL.print("[HTTP] begin...\n");
  // configure traged server and url
  //http.begin("https://www.howsmyssl.com/a/check", ca); //HTTPS

 float t = sht31.readTemperature();
float h = sht31.readHumidity();

sprintf(buf,"https://api.thingspeak.com/update?api_key=HARZP4QU1ZOIO2EA&field1=%f&field2=%
f",t,h);

 http.begin(buf);  //HTTP
   USE_SERIAL.print("[HTTP] GET...\n");
 // start connection and send HTTP header
  int httpCode = http.GET();

   // httpCode will be negative on error
   if (httpCode > 0) {
     // HTTP header has been send and Server response header has been handled
    USE_SERIAL.printf("[HTTP] GET... code: %d\n", httpCode);

   // file found at server
  if (httpCode == HTTP_CODE_OK) {
  String payload = http.getString();
    USE_SERIAL.println(payload);
    }
 } else {
     USE_SERIAL.printf("[HTTP] GET... failed, error: %s\n",
http.errorToString(httpCode).c_str());
   }
    http.end();
  }
  delay(5000);
}
```

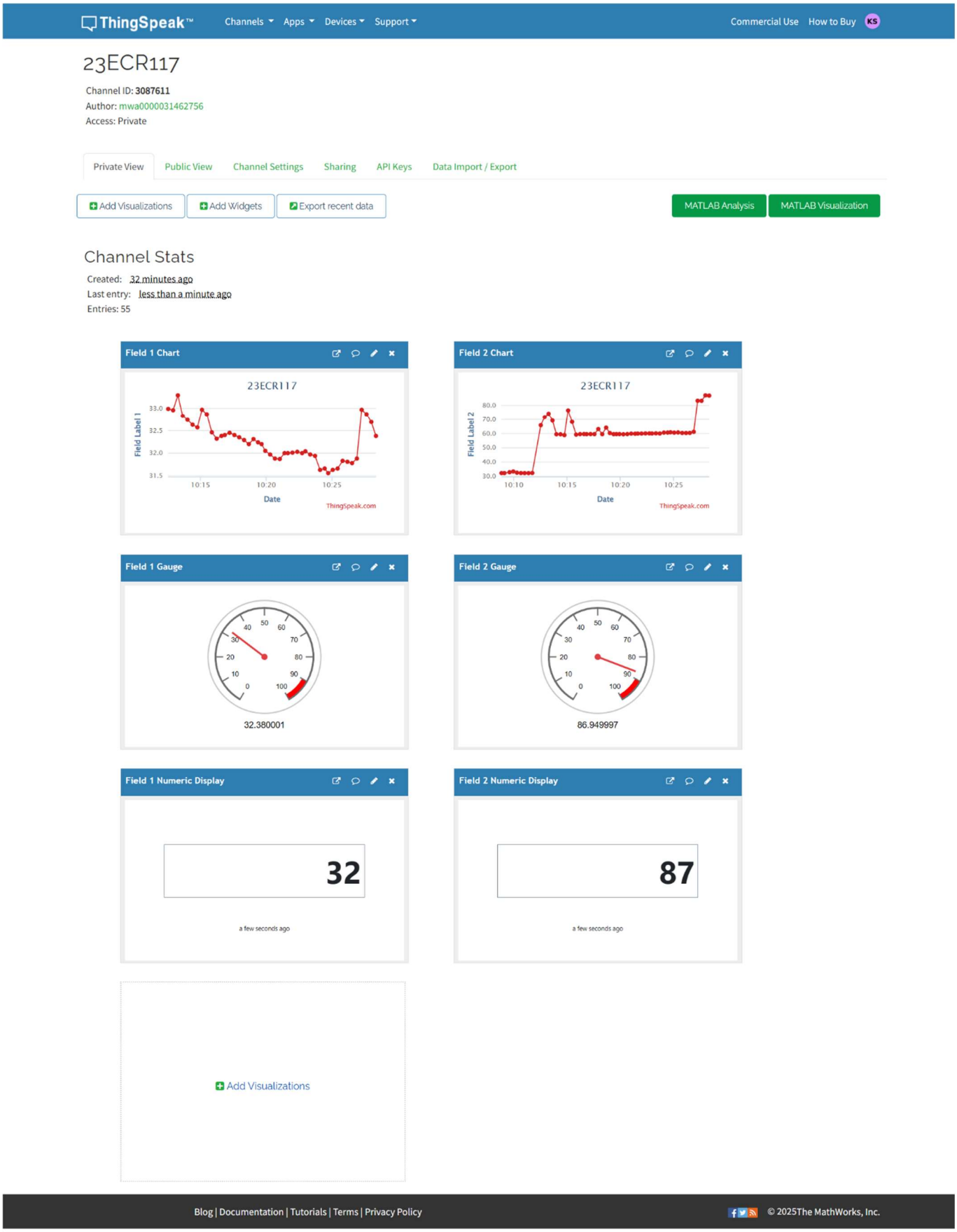
Hardware Output :



ThingSpeak Cloud Output :


```
      <field1>31.809999</field1>
      <field2>60.389999</field2>
    </feed>
  ▼<feed>
      <created-at type="dateTime">2025-09-24T04:56:32Z</created-at>
      <entry-id type="integer">50</entry-id>
      <field1>31.780001</field1>
      <field2>60.459999</field2>
    </feed>
  ▼<feed>
      <created-at type="dateTime">2025-09-24T04:56:54Z</created-at>
      <entry-id type="integer">51</entry-id>
      <field1>31.879999</field1>
      <field2>61.180000</field2>
    </feed>
  ▼<feed>
      <created-at type="dateTime">2025-09-24T04:57:16Z</created-at>
      <entry-id type="integer">52</entry-id>
      <field1>32.970001</field1>
      <field2>83.379997</field2>
    </feed>
  ▼<feed>
      <created-at type="dateTime">2025-09-24T04:57:38Z</created-at>
      <entry-id type="integer">53</entry-id>
      <field1>32.869999</field1>
      <field2>83.330002</field2>
    </feed>
  ▼<feed>
      <created-at type="dateTime">2025-09-24T04:58:00Z</created-at>
      <entry-id type="integer">54</entry-id>
      <field1>32.689999</field1>
      <field2>87.129997</field2>
    </feed>
  ▼<feed>
      <created-at type="dateTime">2025-09-24T04:58:21Z</created-at>
      <entry-id type="integer">55</entry-id>
      <field1>32.380001</field1>
      <field2>86.949997</field2>
    </feed>
  </feeds>
</channel>
```

ThingSpeak Channel Output :



QR Code :

Rubrics		Marks
Conduct of Experiment	Analyse the problem and develop programming constructs (15)	
	Completeness of the experiment (5)	
Observation/ Record (30)	Interpretation of the findings (15)	
	Simulation and Hardware (5)	
	Adherence to record submission deadline (5)	
	Presentation and completion of record (5)	
Viva (10)	Ability to recall the theoretical concepts	
Total (60)		

Result :

Sending the sensor data to cloud using thingzkit mini through wifi was done successfully.

EX NO : 07	Integrate a third-party application server for data storage and monitoring. (LoRaWAN Communication)	Batch No : 02
DATE :		Roll No : 23ECR117

AIM :

To design and develop an embedded C program to integrate a third-party application server for data storage and monitoring using LoRaWAN Communication. (Switch status monitoring).

Software Required :

Arduino IDE

Hardware Required :

ThingZMate Kit,USB cable

Theory :

Building a thingZmate / The Things Network (TTN) login system typically refers to creating a way to authenticate devices (LoRaWAN nodes) with thingZmate / The Things Network (TTN) to securely send and receive data over a LoRaWAN network. TTN is an open-source, decentralized IoT network that uses LoRaWAN for communication between low-power devices and gateways. For devices to communicate with TTN, they must authenticate using specific credentials. LoRaWAN in India typically uses the 865 MHz to 867 MHz band for LoRa communications.

Procedure :

Here's a basic step-by-step guide on how to set up a device and authenticate it with **The Things Network (TTN) :** Assume the LoRa gateway modem is already configured to the TTN network. In this experiment, Class A communication mode and Over-the-Air Activation (OTAA) Authentication is recommended.

Steps to Build a TTN Login (Authentication) System :

1. **Create a TTN Account**
   - Go to the TTN Console: Visit The Things Network Console. (https://accounts.thethingsindustries.com)
   - Sign up or log in if you already have an account.
2. **Create a New Application in TTN**
   - Once logged in to the console, navigate to the Applications section.
   - Click on the "Add Application" button.
   - Provide a name and description for your application.
   - Select the Region that best matches your location (e.g., India).
   - After creating the application, you'll be taken to the application dashboard.
3. **Create a New Device (Join)**
   - Inside the application dashboard, go to the Devices section.
   - Click on "Add Device" to register your LoRaWAN device.
   - You'll be prompted to enter details like the Device ID (a unique identifier for the device), Device EUI, and App Key.

   The device can authenticate using one of the following methods :

• OTAA (Over-the-Air Activation): This mode allows for the device to join the network dynamically.

- The device will authenticate automatically using the Device EUI, Application EUI, and App Key.

3.1 Using OTAA Authentication (Recommended for most cases)

• **Device EUI:** This is a unique identifier for your device, usually printed on the device itself or available through your device's firmware.

• **Application EUI:** TTN generates this when you create the application. You will find it in the TTN Console under the application's settings.

• **App Key:** This is a secret key used to authenticate the device during OTAA. It's also available in the TTN Console in the application settings.

• Once these values are added to the TTN device, TTN will manage the authentication automatically when your device attempts to connect.

4. Configure Your Device (LoRaWAN Module)

• Given program sketch and AT command should be uploaded before doing the following procedure.

• Open serial monitor console of Arduino IDE and configure the baud rate as 115200 and set line ending as newline.

• The next step is to configure LoRaWAN device with the credentials that obtained from TTN.

For thingZkit mini WDM module need to :

Set the Device EUI and App Key (for OTAA) AT Commands :

AT Commands needed at Initial to store Keys via ESP32 serial port.

- **AT Commands to set initially (Mandatory) :**
  - FFD // To do factory data reset
  - C01 // To set message type (C01-Conformed/C00-Unconfirmed)
  - R01 // To enable the ADR
  - T1440 // To set the default sampling interval as 1440 minutes (24Hrs) - (Should not give below 5 minutes)
- **If you set the NJM-N01 you should follow the given steps below:**
  - N01 // To set the Activation Type OTAA Mode
  - DXXXXXXXXXXXXXXXX (Device EUI) // To set Device EUI key
  - JXXXXXXXXXXXXXXXX (App EUI) // To set APP EUI key
  - AXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX (AppKey)// To set APP Key
  - ZFF (ATZ) // To take effective action on below settings (As like saving)

CIRCUIT DIAGRAM :

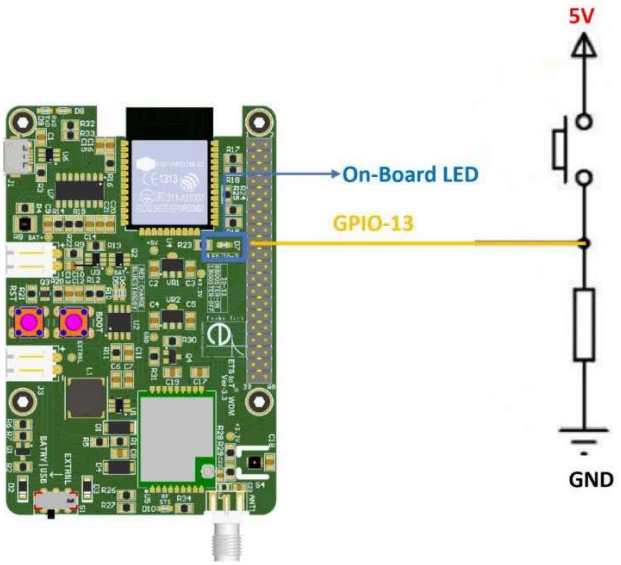


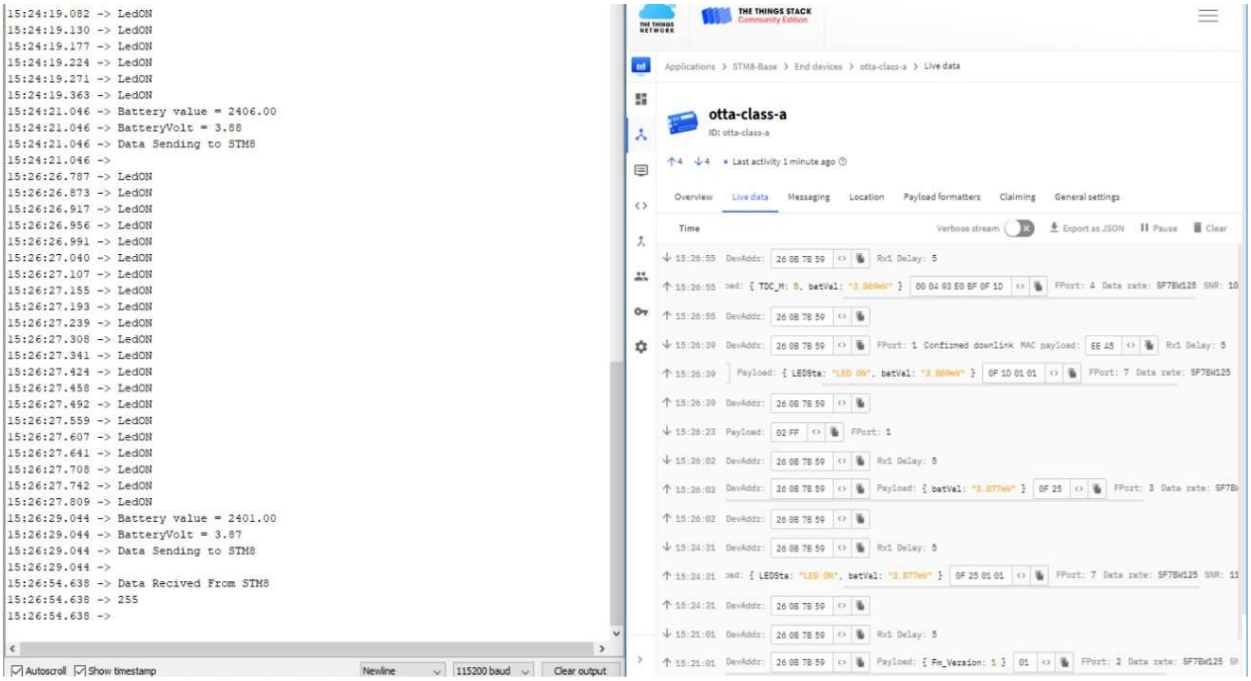
Fig1: SWITH & LED interfacing with ETS_IOT_WDM_v3.3/ ESP32

Code :

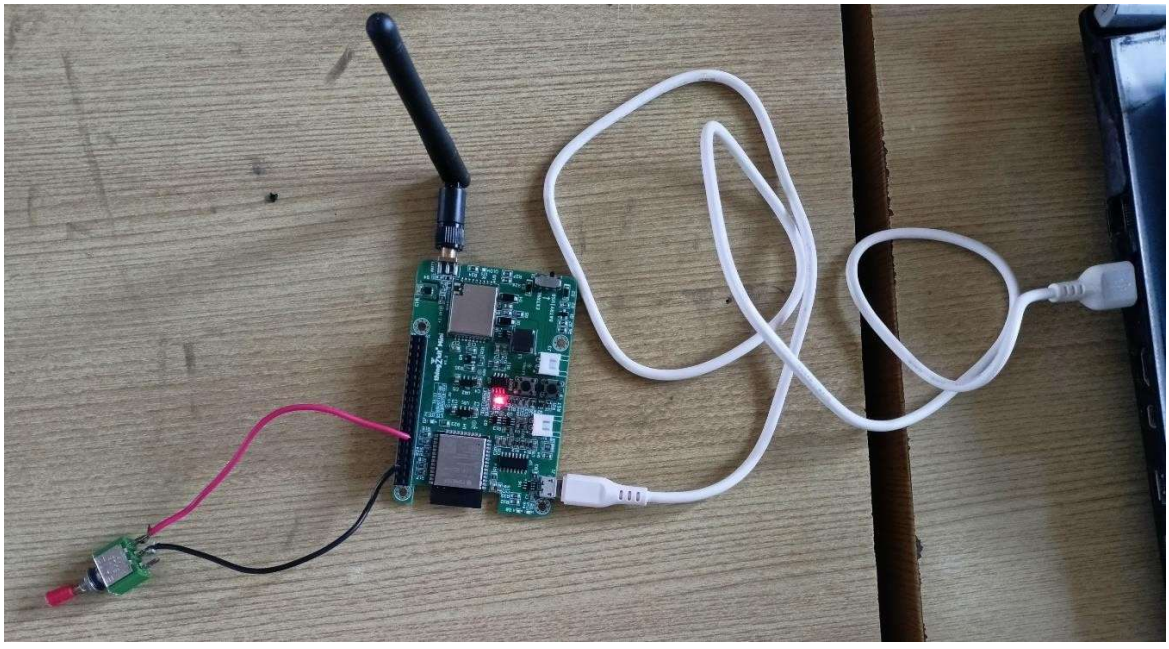
```
/*01 - SWITCH AND LED INTERFACING WITH ETS_WDM_v1.3*/

#if CONFIG_FREERTOS_UNICORE
#define ARDUINO_RUNNING_CORE 0
#else
#define ARDUINO_RUNNING_CORE 1
#endif

#include"AT_Command.h"
#include "BatterRead.h"
#define led       2
#define Button    13

#define fun_Code    1 //Function Code

/* limit of transmit Bytes are 25 */
unsigned char TXBuffer[] = {0,0, fun_Code, 0};
bool buttFlag = 0;
extern bool AT_Flag;

void TaskUplink(void *pvParameters);  // This is a task.
void TaskReadSW(void *pvParameters);  // This is a task.
void UARTsetup(void);//ESP32 to STM8 UART setup function
void UserTXFun( unsigned char *tx, int Size);

void setup()
{
  Serial.begin(115200);
  UARTsetup();
```

```
  xTaskCreatePinnedToCore(TaskUplink,  "TaskUplink",  10240,  NULL,  4,  NULL,  ARDUINO_RUNNING_CORE);
  xTaskCreatePinnedToCore(TaskReadSW,  "TaskReadSW",  10240,  NULL,  3,  NULL,  ARDUINO_RUNNING_CORE);
  pinMode(led, OUTPUT);//Set pin 02 as digital output pin
  pinMode(Button, INPUT);//Set pin 13 as digital input pin

}

void loop() {

}

void TaskUplink(void *pvParameters)  // This is a task.
{
  (void) pvParameters;
  for (;;)
  {
    if (buttFlag == 1 && AT_Flag != 1 && joinFlag==1) {
        if (Class_A == 1) {
          BatteryRead();//Read the Battery voltage from ESP32
          TXBuffer[0] = BatRed >> 8;
          TXBuffer[1] = BatRed;
        }
      /* Transmit Function From ESP32 to STM8 */
      UserTXFun(TXBuffer, sizeof(TXBuffer));// User TXBuffer and Buffer size...
      for (int i = 0; i < 1000; i++)
      {
        ;
      }
      memset(TXBuffer, 0, sizeof (TXBuffer));
      TXBuffer[2] = fun_Code;//Here add your experiment number...
      buttFlag = 0;
    }
    vTaskDelay(4000);
  }
}

void TaskReadSW(void *pvParameters)  // This is a task.
{
  (void) pvParameters;
  for (;;)
  {
    bool Read = 0;
    Read =  digitalRead(Button);
    if (Read == 1) {
      digitalWrite(led, HIGH);
      Serial.println("LedON");
      TXBuffer[3] = 0x01;
      buttFlag = 1;
    }
    if (Read == 0) {
      digitalWrite(led, LOW);
    }
    vTaskDelay(50);
  }
}

```

Simulation Output :



Hardware Output :



Rubrics		Marks
Conduct of Experiment	Analyse the problem and develop programming constructs (15)	
	Completeness of the experiment (5)	
Observation/ Record (30)	Interpretation of the findings (15)	
	Simulation and Hardware (5)	
	Adherence to record submission deadline (5)	
	Presentation and completion of record (5)	
Viva (10)	Ability to recall the theoretical concepts	
Total (60)		

Result :

Thus, the design and development of an embedded C program to integrate a third-party application server for data storage and monitoring using LoRaWAN Communication was done successfully

EX NO : 08	Integrate a third-party application server for controlling the motor using the WDM board. (LoRaWAN Communication)	Batch No : 02
DATE :		Roll No : 23ECR117

AIM :

To design and develop an embedded C program to integrate a third-party application server for controlling the motor using the WDM board using LoRaWAN Communication.

Software Required :

Arduino IDE

Hardware Required :

Sl. No	Apparatus name	Range	Quantity
1.	thingZkit Mini	-	1
2.	USB data cable	-	1
3.	DC Stepper Motor	-	1
4.	L298N (or) UL2003 Motor Driver	-	1
5.	DC +12V Power Supply	-	1
6.	Connecting wires	-	As required

Theory :

Building a thingZmate / The Things Network (TTN) login system typically refers to creating a way to authenticate devices (LoRaWAN nodes) with thingZmate / The Things Network (TTN) to securely send and receive data over a LoRaWAN network. TTN is an open-source, decentralized IoT network that uses LoRaWAN for communication between low-power devices and gateways. For devices to communicate with TTN, they must authenticate using specific credentials. LoRaWAN in India typically uses the 865 MHz to 867 MHz band for LoRa communications.

Procedure :

Here's a basic step-by-step guide on how to set up a device and authenticate it with **The Things Network (TTN) :** Assume the LoRa gateway modem is already configured to the TTN network. In this experiment, Class A communication mode and Over-the-Air Activation (OTAA) Authentication is recommended.

Steps to Build a TTN Login (Authentication) System :

1. **Create a TTN Account**
   - Go to the TTN Console: Visit The Things Network Console. (https://accounts.thethingsindustries.com)
   - Sign up or log in if you already have an account.
2. **Create a New Application in TTN**
   - Once logged in to the console, navigate to the Applications section.
   - Click on the "Add Application" button.
   - Provide a name and description for your application.
   - Select the Region that best matches your location (e.g., India).

• After creating the application, you'll be taken to the application dashboard.

3. Create a New Device (Join)

• Inside the application dashboard, go to the Devices section.

• Click on "Add Device" to register your LoRaWAN device.

• You'll be prompted to enter details like the Device ID (a unique identifier for the device), Device EUI, and App Key.

The device can authenticate using one of the following methods :

• OTAA (Over-the-Air Activation): This mode allows for the device to join the network dynamically.

- The device will authenticate automatically using the Device EUI, Application EUI, and App Key.

3.1 Using OTAA Authentication (Recommended for most cases)

• **Device EUI:** This is a unique identifier for your device, usually printed on the device itself or available through your device's firmware.

• **Application EUI:** TTN generates this when you create the application. You will find it in the TTN Console under the application's settings.

• **App Key:** This is a secret key used to authenticate the device during OTAA. It's also available in the TTN Console in the application settings.

• Once these values are added to the TTN device, TTN will manage the authentication automatically when your device attempts to connect.

4. Configure Your Device (LoRaWAN Module)

• Given program sketch and AT command should be uploaded before doing the following procedure.

• Open serial monitor console of Arduino IDE and configure the baud rate as 115200 and set line ending as newline.

• The next step is to configure LoRaWAN device with the credentials that obtained from TTN.

For thingZkit mini WDM module need to :

Set the Device EUI and App Key (for OTAA) AT Commands :

AT Commands needed at Initial to store Keys via ESP32 serial port.

- **AT Commands to set initially (Mandatory) :**
  - FFD // To do factory data reset
  - C01 // To set message type (C01-Conformed/C00-Unconfirmed)
  - R01 // To enable the ADR
  - T1440 // To set the default sampling interval as 1440 minutes (24Hrs) - (Should not give below 5 minutes)
- **If you set the NJM-N01 you should follow the given steps below:**
  - N01 // To set the Activation Type OTAA Mode

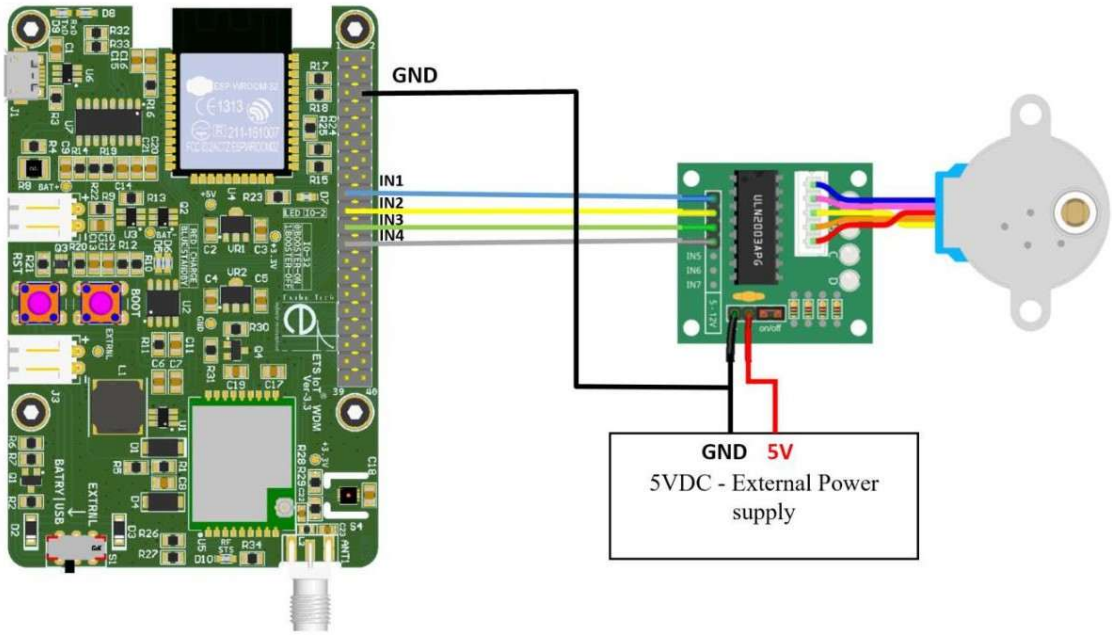
• DXXXXXXXXXXXXXXXX (Device EUI) // To set Device EUI key

• JXXXXXXXXXXXXXXXX (App EUI) // To set APP EUI key

• AXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX (AppKey)// To set APP Key

• ZFF (ATZ) // To take effective action on below settings (As like saving)

CIRCUIT DIAGRAM :



Code :

```
/*Note: use the 5v externalpower supply*/

#if CONFIG_FREERTOS_UNICORE
#define ARDUINO_RUNNING_CORE 0
#else
#define ARDUINO_RUNNING_CORE 1
#endif
#define fun_Code    14 //Function Code

#include "AT_Command.h"
#include "BatterRead.h"
#include <Stepper.h> // Include the header file
#define RX         16
#define TX         17
#define IN1        13
#define IN2        12
#define IN3        14
#define IN4        15

// change this to the number of steps on your motor
#define STEPS 2000

// create an instance of the stepper class using the steps and pins
Stepper stepper (STEPS, IN1, IN2, IN3, IN4); // Create instances in w

unsigned char TXBuffer[] = {0, 0, fun_Code};
uint8_t sendCount = 0;
/*set the flag after received stm8 data*/
bool DowFlag = 0, EnableFlag = 0;
uint8_t RREnCont = 0; // For delay purpose variable
uint16_t value = 0;// For store the Downlink value in this variable

```

```
/* RTOS Function */
void TaskDownlink(void *pvParameters);
void TaskStepper(void *pvParameters);

/* User Fucntion */
void StepperRun(uint16_t rotaVal);

void setup()
{
  Serial.begin(115200);
  UARTsetup();
  xTaskCreatePinnedToCore(TaskStepper,  "TaskStepper",  10240,  NULL,  4,  NULL,  ARDUINO_RUNNING_CORE);
  xTaskCreatePinnedToCore(TaskDownlink,  "TaskDownlink",  10240,  NULL,  3,  NULL,  ARDUINO_RUNNING_CORE);

  /*No uplink & Frist time pass the 0 value */
  unsigned char txValue = 0;
  UserTXFun(&txValue, 1);

}

void loop() {

}

void TaskStepper(void *pvParameters)  // This is a task.
{
  (void) pvParameters;
  for (;;)
  {
    if (Class_A == 1) {
      sendCount++;
    }
    if (sendCount == 60) {
      BatteryRead();//Read the Battery voltage from ESP32
      TXBuffer[0] = BatRed >> 8;
      TXBuffer[1] = BatRed;
      UserTXFun(TXBuffer, sizeof(TXBuffer));// User TXBuffer and Buffer size...
      memset(TXBuffer, 0, sizeof (TXBuffer));
      TXBuffer[2] = fun_Code;
      sendCount = 0;
    }
    StepperRun( value );
    vTaskDelay(1000);
  }
}

void TaskDownlink(void *pvParameters)
{

  (void) pvParameters;
  for (;;)
  {
    /*When the user receives the downlink, the flag will be set.*/
    if (usDwStFlag == 1 && joinFlag == 1) {
      RREnCont++;
    }

void StepperRun(uint16_t rotaVal) {
  /* One complete rotation given value is 520. you will be achieved the proper angle when you change the given value from a network server. */
  if ( EnableFlag == 1) {
    Serial.println(" Enter Stepper is run ");
    for (int i = 0; i < value; i++)
    {
      digitalWrite(IN1, HIGH);
      digitalWrite(IN2, LOW);
      digitalWrite(IN3, LOW);
      digitalWrite(IN4, LOW);
      delay(1);
      digitalWrite(IN1, HIGH);
      digitalWrite(IN2, HIGH);
      digitalWrite(IN3, LOW);
      digitalWrite(IN4, LOW);
      delay(1);
      digitalWrite(IN1, LOW);
      digitalWrite(IN2, HIGH);
      digitalWrite(IN3, LOW);
      digitalWrite(IN4, LOW);
      delay(1);
      digitalWrite(IN1, LOW);
      digitalWrite(IN2, HIGH);
      digitalWrite(IN3, HIGH);
      digitalWrite(IN4, LOW);
      delay(1);
      digitalWrite(IN1, LOW);
      digitalWrite(IN2, LOW);
      digitalWrite(IN3, HIGH);
      digitalWrite(IN4, LOW);
      delay(1);
```

```
      digitalWrite(IN2, LOW);
      digitalWrite(IN3, LOW);
      digitalWrite(IN4, HIGH);
      delay(1);
    }
    digitalWrite(IN1, HIGH);
    digitalWrite(IN2, LOW);
    digitalWrite(IN3, LOW);
    digitalWrite(IN4, LOW);
    delay(100);
    value = 0;
    EnableFlag = 0;

  } else if (DowFlag == 1 ) {
    digitalWrite(IN1, LOW);
    digitalWrite(IN2, LOW);
    digitalWrite(IN3, LOW);
    digitalWrite(IN4, LOW);
    delay(10);
    DowFlag = 0;
    EnableFlag = 1;
  } else {}
}
```

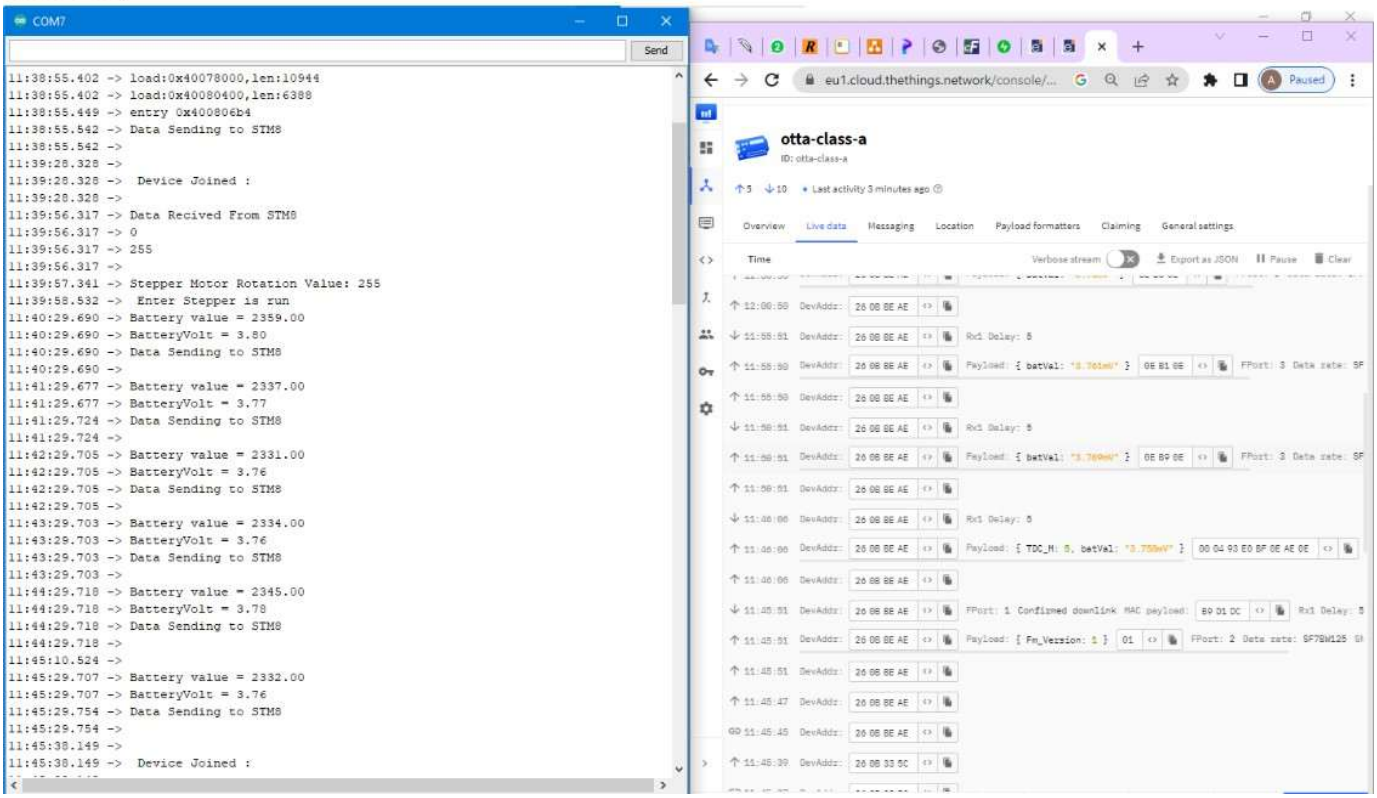
Output Serial Monitor ×

Message (Enter to send message to 'ESP32 Dev Mod

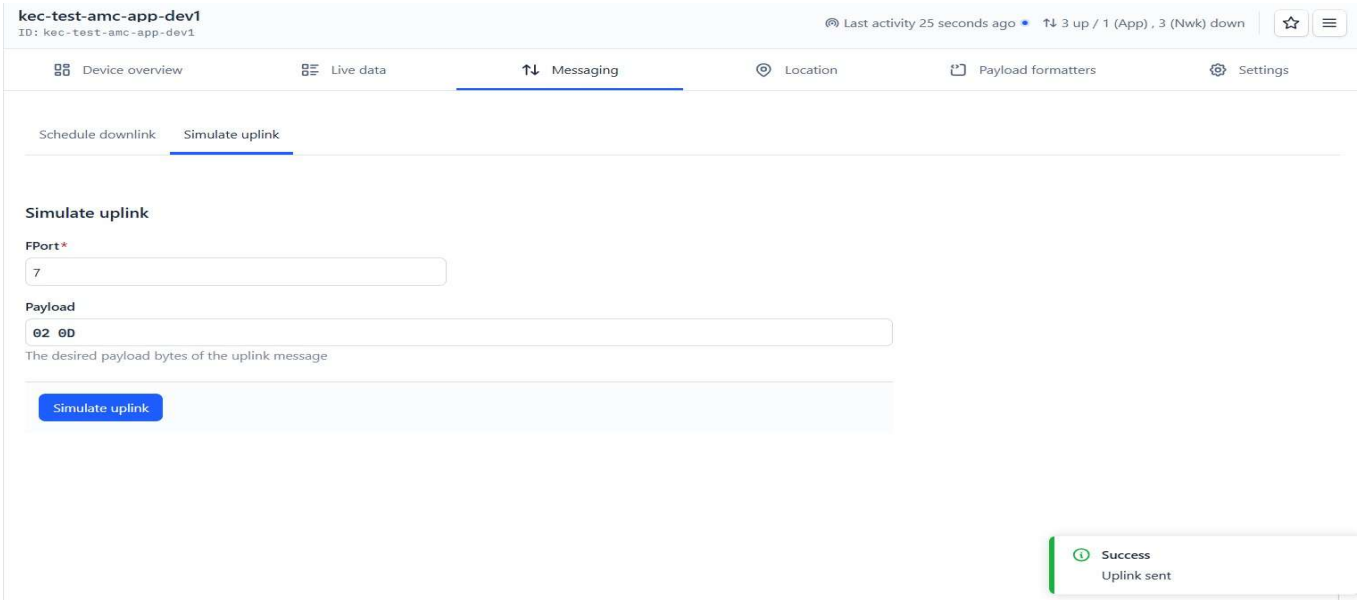
```
load:0x40080400,len:4
load:0x40080404,len:3504
entry 0x400805cc
Data Sending to STM8

Data Recived From STM8
11
```

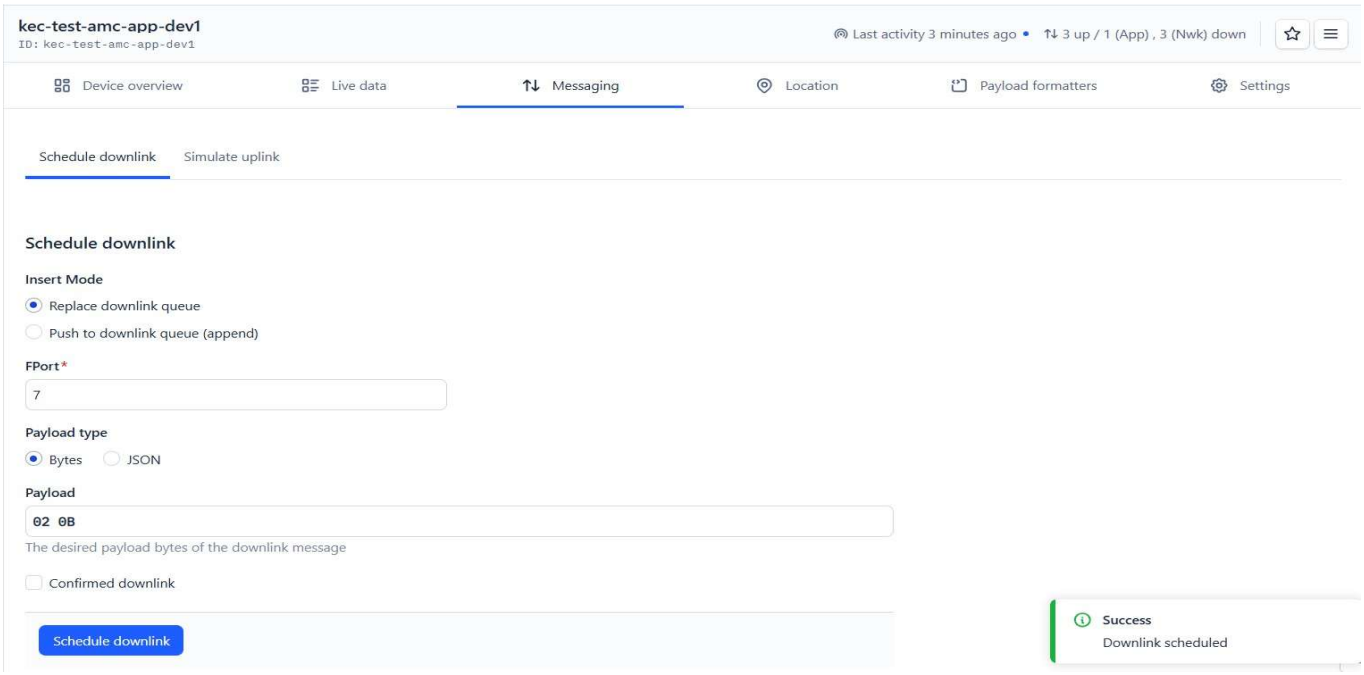
Simulation output :

Status Monitoring

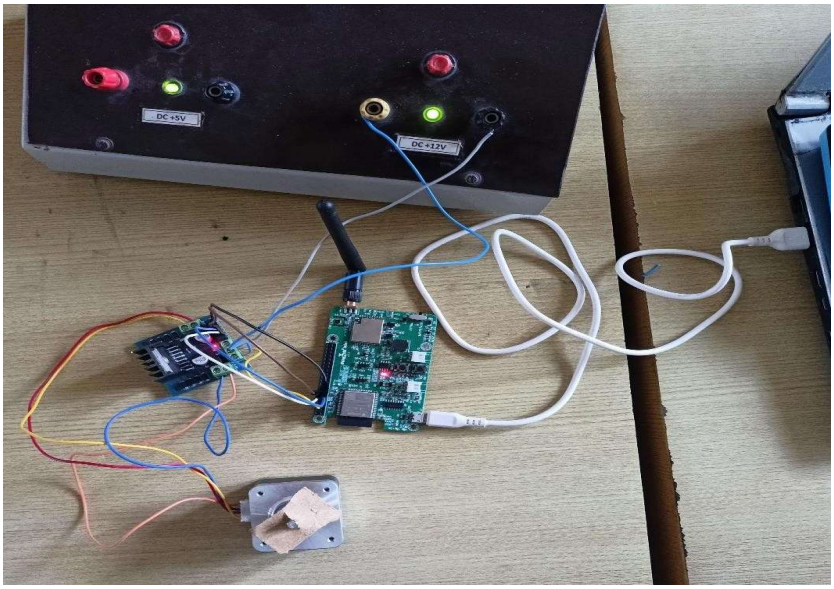


Uplink and Downlink to control DC stepper motor from cloud :





Hardware Output :



Rubrics		Marks
Conduct of Experiment	Analyse the problem and develop programming constructs (15)	
	Completeness of the experiment (5)	
Observation/ Record (30)	Interpretation of the findings (15)	
	Simulation and Hardware (5)	
	Adherence to record submission deadline (5)	
	Presentation and completion of record (5)	
Viva (10)	Ability to recall the theoretical concepts	
Total (60)		

Result :

Thus, the design and development of an embedded C program to integrate a third-party application server for controlling the motor using the WDM board using LoRaWAN Communication was done successfully.